

Test case Generator UI

Generate test cases for fund transfer in Chat and verified the test cases

The screenshot displays the Test Case Generator UI on the left and a network inspector on the right. The UI shows a list of test cases for fund transfer, including steps and expected results. The network inspector shows two requests: a 'chat' request with status 200 and a 'chat' request with status 204.

Name	Status	Type	Initiator	Size	Time	Fulfil...
chat	200	fetch	api.tc.133	2.9 kB	5.65 s	
chat	204	preflight	Preflight	0 B	21 ms	

Verifying the test cases in Backend in Terminal

The screenshot displays the Test Case Generator UI on the left and a network inspector on the right. The UI shows a list of test cases for fund transfer, including steps and expected results. The network inspector shows the response of a 'chat' request, which is a JSON object containing session information and test cases.

```
{
  "sessionId": "758d81fe-58e4-4fd8-904d",
  "domain": "banking",
  "sourceType": "document",
  "testCases": [
    {
      "id": "TC-001",
      "title": "Fund Transfer - Val",
      "steps": [
        "Login to the online bank",
        "Navigate to the fund tra",
        "Enter the recipient's ac",
        "Enter the transfer amou",
        "Confirm the transaction",
        "Verify the OTP/2FA valid"
      ],
      "expectedResult": "The fund t"
    },
    {
      "id": "TC-002",
      "title": "Fund Transfer - Ins",
      "steps": [
        "Login to the online bank",
        "Navigate to the fund tra",
        "Enter the recipient's ac",
        "Enter the transfer amou"
      ]
    }
  ]
}
```

Verified by uploading the PDF document and verified the test cases in the chat

The screenshot shows the Test Case Generator interface on the left and the Network tab on the right. The UI has a 'Domain' dropdown set to 'Banking' and an 'Attach file' button with 'Uploaded: basic-text.pdf'. A 'New Chat' button is in the top right. Below, a sample document text is shown. The 'BANKING' tab is active, displaying three test cases: 'TC-001 PDF Rendering Test', 'TC-002 PDF Pagination Test', and 'TC-003 PDF Content Verification Test'. Each test case includes steps and expected results. The Network tab on the right shows a list of requests: 'upload', 'upload', 'chat', and 'chat'. The 'chat' request is selected, showing a JSON response with 'sessionId', 'domain', 'sourceType', and 'testCases'.

Upload the .png image and verified the test cases generated in UI and in Network tab

The screenshot shows the Test Case Generator interface on the left and the Network tab on the right. The UI has a 'Domain' dropdown set to 'Hospital' and an 'Attach file' button with 'Uploaded: Hospital.png'. A 'New Chat' button is in the top right. Below, a 'Generate test cases based on this UI screenshot' button is visible. The 'HOSPITAL' tab is active, displaying five test cases: 'TC-001 Verify Hospital Name Display', 'TC-002 Verify Hospital Address and Contact Information', 'TC-003 Verify Visual Representation of Hospital Building', 'TC-004 Verify Presence of Navigation or Service Links', and 'TC-005 Verify Accessibility Features'. Each test case includes steps and expected results. The Network tab on the right shows a list of requests: 'upload', 'upload', 'chat', and 'chat'. The 'chat' request is selected, showing a JSON response with 'sessionId', 'domain', 'sourceType', and 'testCases'.

Adding one more test case for the existing test cases

←localhost:5173

4 test cases

Expand allCollapse all

TC-001 PDF Rendering Test

STEPS

1. Open the PDF document in a PDF reader

2. Verify that all text formatting options are rendered correctly (bold, italic, strikethrough)

3. Check that unordered and ordered lists are displayed as expected

4. Confirm that the block quote is formatted correctly

5. Verify that the table is rendered with correct headers and row/column data

EXPECTED RESULT

All text formatting options, lists, block quote, and table are rendered correctly in the PDF reader

TC-002 PDF Pagination Test

STEPS

1. Open the PDF document in a PDF reader

2. Verify that the document is split into 2 pages as indicated by the "... 1 of 2 ..." and "... 2 of 2 ..." markers

3. Check that all content is correctly paginated and no content is missing or duplicated

EXPECTED RESULT

The PDF document is correctly paginated, with all content displayed across 2 pages

TC-003 PDF Content Verification Test

STEPS

1. Open the PDF document in a PDF reader

2. Verify that all expected content is present, including text formatting examples, lists, block quote, and table

3. Check that the sample file information and website URL are correctly displayed

EXPECTED RESULT

All expected content is present and correctly displayed in the PDF reader

TC-004 PDF Search Functionality Test

STEPS

1. Open the PDF document in a PDF reader

2. Use the search function to find specific keywords (e.g. "Sample-Files.com")

3. Verify that the search results are accurate and relevant

EXPECTED RESULT

The search function returns accurate and relevant results for the searched keywords

Describe requirements or ask for refinements...

Send

Network

Filter

Fetch/XHRDocCSSJSFontImgMediaManifestSocketWasmOther

10,000 ms20,000 ms30,000 ms40,000 ms50,000 ms60,000 ms70,000 ms

Name

upload

upload

chat

chat

chat

Headers

Payload

Preview

Response

"steps": [

"Open the PDF document in

"Verify that the document

"Check that all content i

],

"expectedResult": "The PDF do

},

{

"id": "TC-003",

"title": "PDF Content Verific

"steps": [

"Open the PDF document in

"Verify that all expected

"Check that the sample fi

],

"expectedResult": "All expect

},

{

"id": "TC-004",

"title": "PDF Search Function

"steps": [

"Open the PDF document in

"Use the search function "

"Verify that the search r

],

"expectedResult": "The search

},

]

6 requests5.0 kB transferred4.2

Console

Issues

Removing the test case TC004

←localhost:5173

4 test cases

Expand allCollapse all

TC-001 Successful ATM Cash Withdrawal

STEPS

1. Insert a valid ATM card

2. Enter the correct PIN

3. Select the withdrawal option

4. Choose the account type (e.g., checking, savings)

5. Enter a withdrawal amount exceeding the daily limit

6. Confirm the transaction

EXPECTED RESULT

The ATM dispenses the cash, and the account balance is updated accordingly.

TC-002 ATM Card Validation

STEPS

1. Insert an expired ATM card

2. Enter the correct PIN

3. Select the withdrawal option

EXPECTED RESULT

The ATM displays an error message indicating that the card is expired and cannot be used.

TC-003 Insufficient Balance

STEPS

1. Insert a valid ATM card

2. Enter the correct PIN

3. Select the withdrawal option

4. Choose the account type (e.g., checking, savings)

5. Enter a withdrawal amount exceeding the available balance

EXPECTED RESULT

The ATM displays an error message indicating that the account balance is insufficient.

TC-005 Daily Withdrawal Limit

STEPS

1. Insert a valid ATM card

Remove the TC-004

Network

Filter

Fetch/XHRDocCSSJSFontImgMediaManifestSocketWasmOther

500 ms1,000 ms1,500 ms2,000 ms2,500 ms

Name

chat

chat

Headers

Payload

Preview

Response

"steps": [

"Insert an expired ATM ca

"Enter the correct PIN",

"Select the withdrawal op

],

"expectedResult": "The ATM di

},

{

"id": "TC-003",

"title": "Insufficient Balanc

"steps": [

"Insert a valid ATM card"

"Enter the correct PIN",

"Select the withdrawal op

"Choose the account type

"Enter a withdrawal amoun

],

"expectedResult": "The ATM di

},

{

"id": "TC-005",

"title": "Daily Withdrawal Li

"steps": [

"Insert a valid ATM card"

"Enter the correct PIN",

"Select the withdrawal op

],

"expectedResult": "The daily withdrawal li

},

]

2 requests1.7 kB transferred1.4

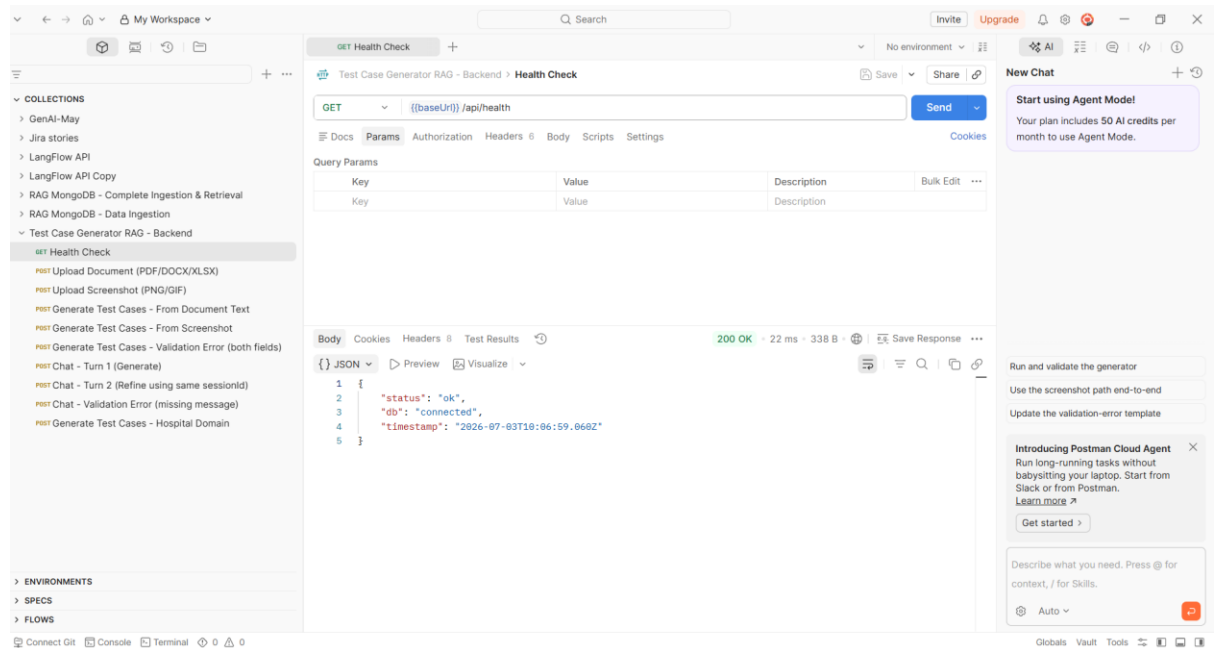
Console

Issues

Postman: -

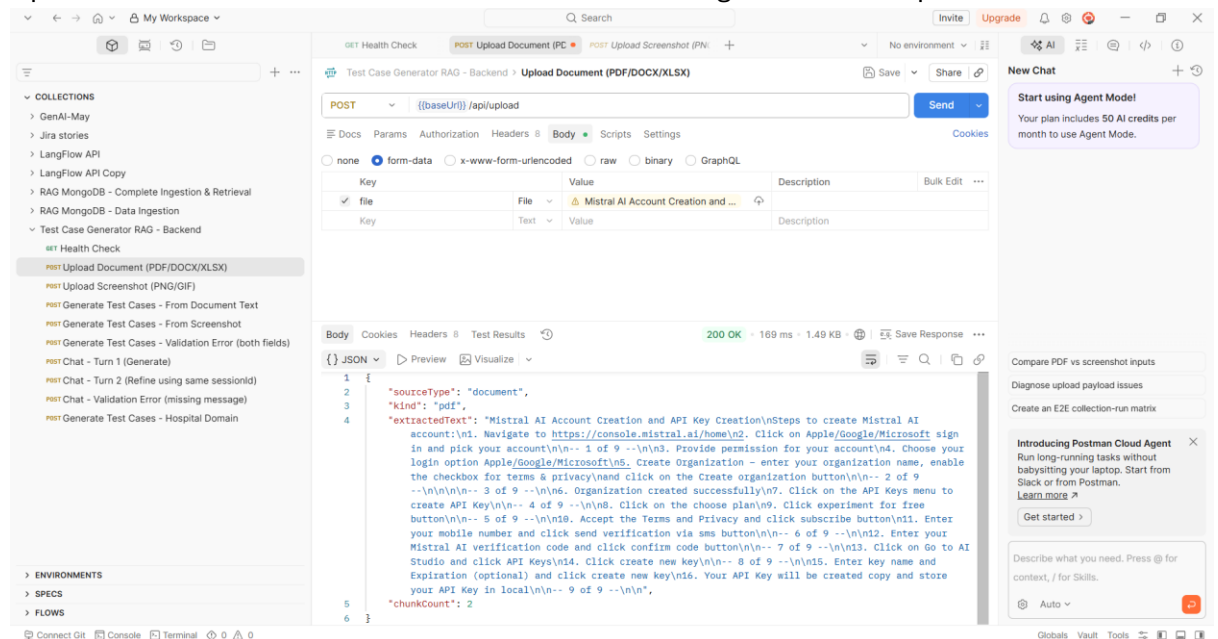
1. HealthCheck: -

Verified the Backend service is connected



2. Upload Document: -

Uploaded the PDF document and verified the test cases generated in response



3. Upload Screenshots: -

Uploading the .png file and verified the base64 for that image is generated in response

The screenshot shows the Postman interface with a workspace named "My Workspace". On the left, a collection named "Test Case Generator RAG - Backend" is expanded, showing a sub-collection "Upload Screenshot (PNG/GIF)". The selected request is a POST to "[(baseUri)]/api/upload". The body is set to "form-data". A file named "Hospital.png" is attached. The response is a 200 OK status with a JSON body containing a base64-encoded image data URL. The response time is 10 ms and the size is 23.72 KB.

```
POST [(baseUri)]/api/upload
form-data
file Hospital.png

{
  "sourceType": "screenshot",
  "kind": "image",
  "mimeType": "image/png",
  "dataUrl": "data:image/png;base64,UK1GPxGAABXRUJQV1A4IABGAQc7WcGASq1AesApp1AmU11o6IqLHVOMUAT1U2U9K1sM1mUz2FNm8QPgNK3/2f12/SS3oPKsY8ByKqoKPLU6pe8/mH0v89PmS8V8Y5E8+11m8ecP84I20801L/eP87-4/qi0CT1v919Vz0vXAB1nB2vQvR4R+BTSp/DUv9YNP4Jw+3091v/ezv+778f2z+q0Av2vFmB7s3QF+dE3Tz2/yf+56J/uv+29gH8zvXP/x+3Lw/8KsF/8/1I/93/LERZ/8+Kz9z/6HSL9MD97fADVfFNVI5S6GvthEP4NdmMgCL3h3VetmHex1AmBvX1CF52/Wbjdz31jKq5Pyv86tkw0z5BFaCoP20y68V5z8neaFCk1BV01H02d5Hm8gM5kcn0JQd7c3HV81aJm1fSd1qLxm1xYaed0AYyERFeN/C-38ZVTL+26AP/2jPmMNV101033/PjYVTOo/EBv8o1JGKPb5Xx19CmfZLQ88GU29d+3gJmM3JAUk1Bp5Nf9tm75JFY0/ht8P3k3x5D50J0m3M17R60C1xym20B1H1hBEVDU1nu1BcmfMKnx2o3k/ngeGPFVakSy4c7KA0fFjnyYmP9m78McIn+XNPMTpewV7baUyM1ind+DLsgTkoFy1sABnM1Bkn8001u/km+y14D/L4143GyoeHpyP+s3JaswR68IVNkNd1h2D+X/BCP3hgCd2gg78Yk3m2Y61fA5c8z0MpkTSLBqxKXf9v8Uz5mkalZ97G0a9Uo+3Bq+b7X7Y9ABcHetz8v1qnpQ2Rvwj7F5x4CLXc1d/HhAH1K7h2/4u92LTUUF0omVmE33FQb3bZLnc8yypYVW1NSy+etsJ0kfea/@+YXyYOVqKRWYHMa831ta11275GAC44+YNArh5Bq2Lw71MhMvL35m599u9EY1PEQh8s3Uf9CpcQ291zPPv1e7eG0t492J081EEKQpM/hd+IE3An8a1zyFAlh7KUPER9R2YF481rr/zEJVz6+TplJ0H4/LwxCOLcRtRm7UYG/35a2un54+o6vmq7+G0su8TbJ/CjDyF
```

4. Generate Test cases – from Screenshot

The screenshot shows the Postman interface with a workspace named "My Workspace". On the left, a collection named "Test Case Generator RAG - Backend" is expanded, showing a sub-collection "Generate Test Cases - From Screenshot". The selected request is a POST to "[(baseUri)]/api/generate-testcases". The body is set to "raw" with JSON content. The response is a 200 OK status with a JSON body containing test cases for a banking application. The response time is 2.51 s and the size is 2.48 KB.

```
POST [(baseUri)]/api/generate-testcases
raw
{
  "domain": "banking",
  "imageDataUrl": "[(base64img)]",
  "additionalContext": "This is the login screen of the mobile banking app."
}

{
  "domain": "banking",
  "sourceType": "screenshot",
  "testCases": [
    {
      "id": "TC-001",
      "title": "Valid Login",
      "steps": [
        "Enter valid username",
        "Enter valid password",
        "Click on login button"
      ],
      "expectedResult": "User is logged in successfully and redirected to the home/dashboard page."
    },
    {
      "id": "TC-002",
      "title": "Invalid Username",
      "steps": [

```

5. Generate test cases – from Document Text

The screenshot shows the Postman interface with a collection named 'Test Case Generator RAG - Backend'. The selected item is 'Generate Test Cases - From Document Text'. The request is a POST to '[[baseUri]]/api/generate-testcases'. The body is raw JSON with the following content:

```
1 {
2   "domain": "banking",
3   "requirementsText": "Fund transfers above INR 50,000 require OTP validation. Daily transfer limit is INR 200,000."
4 }
```

The response is a 200 OK status, 2.06 s, 1.61 KB. The response body is JSON:

```
1 {
2   "testCases": [
3     {
4       "id": "TC-001",
5       "title": "Fund transfer below INR 50,000 without OTP validation",
6       "steps": [
7         "Initiate a fund transfer of INR 40,000",
8         "Verify if OTP validation is not prompted"
9       ],
10      "expectedResult": "Fund transfer is successful without OTP validation"
11    },
12    {
13      "id": "TC-002",
14      "title": "Fund transfer above INR 50,000 with valid OTP validation",
15      "steps": [
16        "Initiate a fund transfer of INR 60,000"
17      ]
18    }
19  ]
20 }
```

The right sidebar shows a 'New Chat' section with a 'Start using Agent Mode!' button and a 'Use Health Check to localize 502' button.

6. Validate error for both fields

The screenshot shows the Postman interface with the same collection and item as before. The request is a POST to '[[baseUri]]/api/generate-testcases'. The body is raw JSON with the following content:

```
1 {
2   "domain": "banking",
3   "requirementsText": "generate test case for fund transfer",
4   "imageDataUrl": "data:image/png;base64,abc"
5 }
```

The response is a 400 Bad Request status, 9 ms, 343 B. The response body is JSON:

```
1 {
2   "error": "Provide exactly one of requirementsText or imageDataUrl"
3 }
```

The right sidebar shows the same 'New Chat' section as before.

7. Turn 1

The screenshot shows the Postman interface for a collection named 'Test Case Generator RAG - Backend'. The selected item is 'Chat - Turn 1 (Generate)'. The request is a POST to '[[baseUrl]] /api/chat' with a body of raw text. The response is a 200 OK status, 3.12 s, 2.66 KB, with a JSON body. The JSON response contains a session ID, domain, source type, and a list of test cases.

```
POST [[baseUrl]] /api/chat
```

```
{
  "domain": "insurance",
  "message": "Claims above the coverage limit must be auto-flagged for manual underwriter review. Premium must be recalculated whenever coverage is modified mid-term."
}
```

```
{
  "sessionId": "481ebca3-2dcd-404e-973e-48ea556cd13e",
  "domain": "insurance",
  "sourceType": "document",
  "testCases": [
    {
      "id": "TC-001",
      "title": "Claims Above Coverage Limit Auto-Flagging",
      "steps": [
        "Create a new policy with a specified coverage limit",
        "Submit a claim that exceeds the coverage limit",
        "Verify the claim is automatically flagged for manual underwriter review"
      ],
      "expectedResult": "The claim is successfully flagged and appears in the underwriter's queue for manual review"
    }
  ],
  "id": "TC-002"
}
```

8. Refine using same session ID

The screenshot shows the Postman interface for the same collection, but the selected item is 'Chat - Turn 2 (Refine using same sessionId)'. The request is a POST to '[[baseUrl]] /api/chat' with a body of raw text. The response is a 200 OK status, 1.74 s, 3.18 KB, with a JSON body. The JSON response is identical to the one in Turn 1, but the session ID is now a variable '[[sessionId]]'.

```
POST [[baseUrl]] /api/chat
```

```
{
  "sessionId": "[[sessionId]]",
  "domain": "insurance",
  "message": "Add one additional edge-case test for a claim filed on the policy expiration date."
}
```

```
{
  "sessionId": "481ebca3-2dcd-404e-973e-48ea556cd13e",
  "domain": "insurance",
  "sourceType": "document",
  "testCases": [
    {
      "id": "TC-001",
      "title": "Claims Above Coverage Limit Auto-Flagging",
      "steps": [
        "Create a new policy with a specified coverage limit",
        "Submit a claim that exceeds the coverage limit",
        "Verify the claim is automatically flagged for manual underwriter review"
      ],
      "expectedResult": "The claim is successfully flagged and appears in the underwriter's queue for manual review"
    }
  ],
  "id": "TC-002"
}
```

9. Validation error (Missing message)

The screenshot shows the Postman interface with a workspace named "My Workspace". The left sidebar lists collections, including "Test Case Generator RAG - Backend". The main panel shows a POST request to "{{\$baseUri}}/api/chat". The request body is a JSON object:

```
{ 1: { 2: "domain": "insurance" 3: } }
```

. The response is a 400 Bad Request with a status of 7 ms and 307 B. The response body is a JSON object:

```
{ 1: { 2: "error": "message is required" 3: } }
```

. The right sidebar shows a "New Chat" section with a "Start using Agent Model!" button and a "Reproduce and inspect the 400" button.

10. Generate test case for Hospital Domain

The screenshot shows the Postman interface with a workspace named "My Workspace". The left sidebar lists collections, including "Test Case Generator RAG - Backend". The main panel shows a POST request to "{{\$baseUri}}/api/generate-testcases". The request body is a JSON object:

```
{ 1: { 2: "domain": "hospital", 3: "requirementsText": "On admission, every patient must be assigned a bed only after triage vitals are recorded. Prescriptions above the standard dosage threshold require pharmacist countersignature before dispensing." 4: } }
```

. The response is a 200 OK with a status of 2.30 s and 1.97 KB. The response body is a JSON object:

```
{ 1: { 2: "domain": "hospital", 3: "sourceType": "document", 4: "testCases": [ 5: { 6: "id": "TC-001", 7: "title": "Patient Admission with Triage Vitals Recorded", 8: "steps": [ 9: "Admit a new patient to the hospital", 10: "Record triage vitals for the patient", 11: "Assign a bed to the patient" 12: ], 13: "expectedResult": "The patient is successfully assigned a bed after triage vitals are recorded" 14: }, 15: { 16: "id": "TC-002", 17: "title": "Patient Admission without Triage Vitals Recorded", 18: "steps": [
```

. The right sidebar shows a "New Chat" section with a "Start using Agent Model!" button and a "Run Health Check with same input" button.